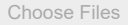


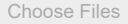
```
import pandas as pd
import numpy as np

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
```

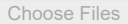
```
from google.colab import files
uploaded = files.upload()
```

 test.csv
test.csv(text/csv) - 3069242 bytes, last modified: 4/4/2026 - 100% done
Saving test.csv to test (1).csv

```
from google.colab import files
uploaded = files.upload()
```

 train.csv
train.csv(text/csv) - 4737878 bytes, last modified: 4/4/2026 - 100% done
Saving train.csv to train (1).csv

```
from google.colab import files
uploaded = files.upload()
```

 sample_submission.csv
sample_submission.csv(text/csv) - 594734 bytes, last modified: 4/4/2026 - 100% done
Saving sample_submission.csv to sample_submission (1).csv

```
import os
os.listdir("/")
```

```
['proc',
 'sys',
 'mnt',
 'tmp',
 'etc',
 'run',
 'bin',
 'boot',
 'usr',
 'sbin',
 'srv',
 'lib32',
 'media',
 'home',
 'lib64',
 'lib',
 'root',
 'var',
 'libx32',
 'opt',
 'dev',
```

```
'kaggle',  
'content',  
'dockerenv',  
'tools',  
'datalab',  
'python-apt',  
'python-apt.tar.xz']
```

```
train = pd.read_csv("train.csv")  
test = pd.read_csv("test.csv")  
sample_submission = pd.read_csv("sample_submission.csv")
```

```
from sklearn.linear_model import Lasso
```

```
X = train.drop("Rings", axis=1)  
y = train["Rings"]
```

```
train = pd.get_dummies(train, columns=["Sex"], drop_first=True)  
test = pd.get_dummies(test, columns=["Sex"], drop_first=True)
```

```
X = train.drop(["Rings", "id"], axis=1)  
y = train["Rings"]
```

```
X_test = test.drop(["id"], axis=1)
```

```
lr = LinearRegression()  
lr.fit(X, y)
```

```
predictions_lr = lr.predict(X_test)
```

```
# Fix negative predictions  
predictions_lr = np.maximum(predictions_lr, 0)
```

```
submission_lr = pd.DataFrame({  
    "id": test["id"],  
    "Rings": predictions_lr  
})
```

```
submission_lr.to_csv("submission_lr.csv", index=False)
```

```
rf = RandomForestRegressor(n_estimators=200, random_state=42)  
rf.fit(X, y)
```

```
predictions_rf = rf.predict(X_test)
```

```
submission_rf = pd.DataFrame({  
    "id": test["id"],  
    "Rings": predictions_rf  
})
```

```
})
```

```
submission_rf.to_csv("submission_rf.csv", index=False)  
submission_rf.head()
```

	id	Rings	
0	90615	10.165	
1	90616	9.800	
2	90617	10.045	
3	90618	11.195	
4	90619	7.830	

Next steps:

[Generate code with submission_rf](#)[New interactive sheet](#)

Applied Question #10 - Carseats Multiple Regression

!pip install ISLP

```
Collecting formulaic from pytorch-lightning:  
  Downloading formulaic-1.2.1-py3-none-any.whl.metadata (7.0 kB)  
Requirement already satisfied: progressbar2<5,>=4.2.0 in /usr/local/lib/python3.12/dist-packages (from pygam->ISLP) (4.5.0)  
Requirement already satisfied: tqdm>=4.57.0 in /usr/local/lib/python3.12/dist-packages (from pytorch_lightning->ISLP) (4.67.3)  
Requirement already satisfied: PyYAML>5.4 in /usr/local/lib/python3.12/dist-packages (from pytorch_lightning->ISLP) (6.0.3)
```

```
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]>=2022.5.0->pytorch)
Requirement already satisfied: idna>=2.0 in /usr/local/lib/python3.12/dist-packages (from yarl<2.0,>=1.17.0->aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]>=2022.5)
Downloading islp-0.4.1-py3-none-any.whl (16.6 MB)
16.6/16.6 MB 71.2 MB/s eta 0:00:00
Downloading lifelines-0.30.3-py3-none-any.whl (409 kB)
409.1/409.1 kB 30.5 MB/s eta 0:00:00
Downloading pygam-0.12.0-py3-none-any.whl (84 kB)
84.6/84.6 kB 7.8 MB/s eta 0:00:00
Downloading pytorch_lightning-2.6.1-py3-none-any.whl (857 kB)
857.3/857.3 kB 49.1 MB/s eta 0:00:00
Downloading torchmetrics-1.9.0-py3-none-any.whl (983 kB)
983.4/983.4 kB 54.7 MB/s eta 0:00:00
Downloading formulaic-1.2.1-py3-none-any.whl (117 kB)
117.3/117.3 kB 8.1 MB/s eta 0:00:00
Downloading lightning_utilities-0.15.3-py3-none-any.whl (31 kB)
Downloading interface_meta-1.3.0-py3-none-any.whl (14 kB)
Building wheels for collected packages: autograd-gamma
Building wheel for autograd-gamma (setup.py) ... done
Created wheel for autograd-gamma: filename=autograd_gamma-0.5.0-py3-none-any.whl size=4030 sha256=cfa4782adab25f60d1086971c404156163d5bae29c5e4bd2baea4fd28e
Stored in directory: /root/.cache/pip/wheels/50/37/21/0a719b9d89c635e89ff24bd93b862882ad675279552013b2fb
Successfully built autograd-gamma
Installing collected packages: lightning-utilities, interface-meta, autograd-gamma, torchmetrics, pygam, formulaic, pytorch_lightning, lifelines, ISLP
Successfully installed ISLP-0.4.1 autograd-gamma-0.5.0 formulaic-1.2.1 interface-meta-1.3.0 lifelines-0.30.3 lightning-utilities-0.15.3 pygam-0.12.0 pytorch-
```

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt

from ISLP import load_data
from statsmodels.stats.anova import anova_lm
from statsmodels.stats.outliers_influence import OLSInfluence
```

```
Carseats = load_data("Carseats")
Carseats.head()
```

	Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc	Age	Education	Urban	US	
0	9.50	138	73	11	276	120	Bad	42	17	Yes	Yes	
1	11.22	111	48	16	260	83	Good	65	10	Yes	Yes	
2	10.06	113	35	10	269	80	Medium	59	12	Yes	Yes	
3	7.40	117	100	4	466	97	Medium	55	14	Yes	Yes	
4	4.15	141	64	3	340	128	Bad	38	13	Yes	No	

Next steps: [Generate code with Carseats](#) [New interactive sheet](#)

```
model1 = smf.ols("Sales ~ Price + Urban + US", data=Carseats).fit()
print(model1.summary())
```

OLS Regression Results

```

=====
Dep. Variable:      Sales      R-squared:      0.239
Model:              OLS       Adj. R-squared:   0.234
Method:             Least Squares  F-statistic:    41.52
Date:              Sun, 05 Apr 2026  Prob (F-statistic): 2.39e-23
Time:              15:18:48    Log-Likelihood: -927.66
No. Observations:   400       AIC:             1863.
Df Residuals:       396       BIC:             1879.
Df Model:           3
Covariance Type:    nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	13.0435	0.651	20.036	0.000	11.764	14.323
Urban[T.Yes]	-0.0219	0.272	-0.081	0.936	-0.556	0.512
US[T.Yes]	1.2006	0.259	4.635	0.000	0.691	1.710
Price	-0.0545	0.005	-10.389	0.000	-0.065	-0.044

```

=====
Omnibus:            0.676    Durbin-Watson:      1.912
Prob(Omnibus):      0.713    Jarque-Bera (JB):    0.758
Skew:               0.093    Prob(JB):            0.684
Kurtosis:           2.897    Cond. No.            628.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
anova_lm(model1)
```

	df	sum_sq	mean_sq	F	PR(>F)
Urban	1.0	0.756615	0.756615	0.123767	7.251713e-01
US	1.0	100.846148	100.846148	16.496407	5.877444e-05
Price	1.0	659.837263	659.837263	107.936143	1.609917e-22
Residual	396.0	2420.834671	6.113219	NaN	NaN

```

model2 = smf.ols("Sales ~ Price + US", data=Carseats).fit()
print(model2.summary())

```

OLS Regression Results

```

=====
Dep. Variable:      Sales      R-squared:      0.239
Model:              OLS       Adj. R-squared:   0.235
Method:             Least Squares  F-statistic:    62.43
Date:              Sun, 05 Apr 2026  Prob (F-statistic): 2.66e-24
Time:              15:18:48    Log-Likelihood: -927.66
No. Observations:   400       AIC:             1861.
Df Residuals:       397       BIC:             1873.
Df Model:           2
Covariance Type:    nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	13.0435	0.651	20.036	0.000	11.764	14.323
Urban[T.Yes]	-0.0219	0.272	-0.081	0.936	-0.556	0.512
US[T.Yes]	1.2006	0.259	4.635	0.000	0.691	1.710
Price	-0.0545	0.005	-10.389	0.000	-0.065	-0.044

```

-----
Intercept    13.0308    0.631    20.652    0.000    11.790    14.271
US[T.Yes]    1.1996    0.258    4.641    0.000    0.692    1.708
Price        -0.0545    0.005   -10.416    0.000   -0.065   -0.044
=====
Omnibus:                0.666   Durbin-Watson:                1.912
Prob(Omnibus):          0.717   Jarque-Bera (JB):          0.749
Skew:                   0.092   Prob(JB):                  0.688
Kurtosis:               2.895   Cond. No.                  607.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

print("Model 1 R-squared:", model1.rsquared)
print("Model 1 Adjusted R-squared:", model1.rsquared_adj)
print("Model 1 AIC:", model1.aic)
print("Model 1 BIC:", model1.bic)

```

```
print()
```

```

print("Model 2 R-squared:", model2.rsquared)
print("Model 2 Adjusted R-squared:", model2.rsquared_adj)
print("Model 2 AIC:", model2.aic)
print("Model 2 BIC:", model2.bic)

```

```

Model 1 R-squared: 0.23927539218405558
Model 1 Adjusted R-squared: 0.2335123269733287
Model 1 AIC: 1863.3120738106174
Model 1 BIC: 1879.2779319990493

```

```

Model 2 R-squared: 0.23926288842678567
Model 2 Adjusted R-squared: 0.23543045965311715
Model 2 AIC: 1861.3186484129808
Model 2 BIC: 1873.2930420543048

```

```
model2.conf_int(alpha=0.05)
```

	0	1
Intercept	11.79032	14.271265
US[T.Yes]	0.69152	1.707766
Price	-0.06476	-0.044195

```
influence = OLSInfluence(model2)
```

```

summary_frame = influence.summary_frame()
summary_frame[["standard_resid", "hat_diag", "cooks_d"]].head()

```

	standard_resid	hat_diag	cooks_d	
0	0.733148	0.003922	0.000705	
1	0.614748	0.009004	0.001145	
2	0.076422	0.009954	0.000020	
3	-0.627878	0.005636	0.000745	
4	-0.775599	0.007928	0.001602	

```
print("Largest absolute standardized residuals:")
print(summary_frame["standard_resid"].abs().sort_values(ascending=False).head(10))

print()
print("Largest leverage values:")
print(summary_frame["hat_diag"].sort_values(ascending=False).head(10))

print()
print("Largest Cook's distance values:")
print(summary_frame["cooks_d"].sort_values(ascending=False).head(10))
```

Largest absolute standardized residuals:

```
376    2.865082
50     2.811022
68     2.622678
25     2.581001
209    2.563363
28     2.418794
352    2.376116
57     2.335540
49     2.321404
297    2.230292
```

Name: standard_resid, dtype: float64

Largest leverage values:

```
42     0.043338
174    0.029687
165    0.028567
125    0.025966
367    0.023707
313    0.023165
171    0.021014
269    0.019195
272    0.018687
356    0.018279
```

Name: hat_diag, dtype: float64

Largest Cook's distance values:

```
25     0.026109
367    0.024287
49     0.022835
316    0.020470
165    0.019755
376    0.018282
```

```

258    0.017677
143    0.016857
30     0.015799
209    0.015663
Name: cooks_d, dtype: float64

```

```

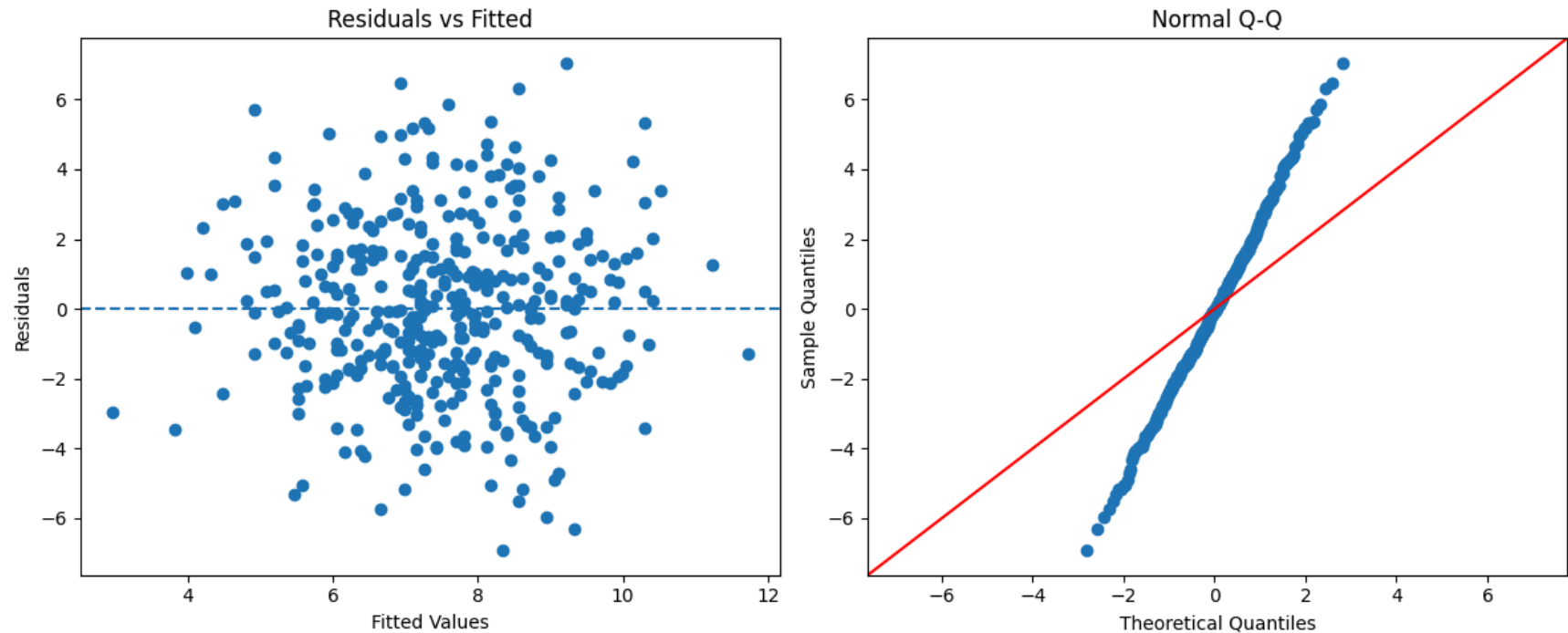
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].scatter(model2.fittedvalues, model2.resid)
axes[0].axhline(0, linestyle="--")
axes[0].set_xlabel("Fitted Values")
axes[0].set_ylabel("Residuals")
axes[0].set_title("Residuals vs Fitted")

sm.qqplot(model2.resid, line="45", ax=axes[1])
axes[1].set_title("Normal Q-Q")

plt.tight_layout()
plt.show()

```



```

X = train.drop("Rings", axis=1)
y = train["Rings"]

```

```

from sklearn.model_selection import train_test_split

```



```
X = train.drop("Rings", axis=1)
y = train["Rings"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
X_train = pd.get_dummies(X_train)
X_test = pd.get_dummies(X_test)
```

```
lin_model = LinearRegression()

lin_model.fit(X_train, y_train)

y_pred_lin = lin_model.predict(X_test)
```

```
lin_model = LinearRegression()

lin_model.fit(X_train, y_train)

y_pred_lin = lin_model.predict(X_test)
```

```
from sklearn.metrics import mean_squared_error, r2_score

mse_lin = mean_squared_error(y_test, y_pred_lin)
r2_lin = r2_score(y_test, y_pred_lin)

print("Linear Regression MSE:", mse_lin)
print("Linear Regression R2:", r2_lin)
```

```
Linear Regression MSE: 4.095480582078631
Linear Regression R2: 0.6013104882981912
```

```
lasso_model = Lasso(alpha=0.1)

lasso_model.fit(X_train, y_train)

y_pred_lasso = lasso_model.predict(X_test)
```

```
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

print("Lasso MSE:", mse_lasso)
print("Lasso R2:", r2_lasso)
```

```
Lasso MSE: 6.207212506332233
Lasso R2: 0.3957362332498432
```

```
rf_model = RandomForestRegressor(random_state=42)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)
```

```
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print("Random Forest MSE:", mse_rf)
print("Random Forest R2:", r2_rf)
```

```
Random Forest MSE: 3.6222297743199245
Random Forest R2: 0.6473808162307539
```

```
print("Model Comparison")

print("Linear Regression R2:", r2_lin)
print("Lasso R2:", r2_lasso)
print("Random Forest R2:", r2_rf)
```

```
Model Comparison
Linear Regression R2: 0.6013104882981912
Lasso R2: 0.3957362332498432
Random Forest R2: 0.6473808162307539
```

```
results = pd.DataFrame({
    "Model": ["Linear Regression", "Lasso", "Random Forest"],
    "R2": [r2_lin, r2_lasso, r2_rf],
    "MSE": [mse_lin, mse_lasso, mse_rf]
})
```

results

	Model	R2	MSE	
0	Linear Regression	0.601310	4.095481	
1	Lasso	0.395736	6.207213	
2	Random Forest	0.647381	3.622230	

Next steps:

[Generate code with results](#)[New interactive sheet](#)

